

MDSA D209 Data Mining I

Task 2 Rev3 Predictive Analysis

Justin Jordan

7/12/2025

WGU

Part I: Research Question

A. Describe the purpose of this data mining by doing the following:

1. Propose one question relevant to a real-world organizational situation that you will answer using one of the following prediction methods:
 - decision trees
 - random forests
 - advanced regression (i.e., lasso or ridge regression)
 - What factors contribute to a customer's tenure based on the available data within the churn dataset? I believe this is an important question to an organization as it costs 10 times more to acquire a new customer than to retain an existing one(Churn Data Consideration and Dictionary 2024). We will be using decision trees to help answer this question.
2. Define one goal of the data analysis. Ensure that your goal is reasonable within the scope of the scenario and is represented in the available data.
 - The goal of this analysis is to identify factors that contribute to a customer's tenure based upon factors within the dataset. If we are able to identify factors that contribute to a customer's tenure, the company can have a strategy in place to help increase a customer's tenure and reduce churn.

Part II: Method Justification

B. Explain the reasons for your chosen prediction method from part A1 by doing the following:

1. Explain how the prediction method you chose analyzes the selected data set. Include expected outcomes.
 - Decision Tree Regression is a method used to predict continuous values like prices or scores by using a tree-like structure. It works by splitting the data into smaller parts based on simple rules taken from the input features. These splits help reduce errors in prediction. At the end of each branch, called a leaf node the model gives a prediction usually the average value of that group.(GeeksforGeeks, Decision tree regression using sklearn - python 2025) The prediction of the target variable(Tenure) will be the average value stored in that leaf node. The expected outcome would be the predicted value for the new data point will be the average of the target variable(Tenure) values of all the training data points that fall into that leaf node. Decision trees are also easy to visualize and interpret.
2. Summarize one assumption of the chosen prediction method.
 - One assumption of the decision tree is that there is independence among the variables. Another assumption is that the data can be split into subsets based on the input variables from the data features.(Ttcbest, 2023)
3. List the packages or libraries you have chosen for Python and justify how each item on the list supports the analysis.
 - In this analysis, we will be using Python within a Jupyter notebook. This gives us the ease and flexibility to have code and markdown cells in one easy to follow document. Python combined with Jupyter make it easy for not only implementing and reading code, but also creating the visual representations of the code all in one place making it easier to understand the data that is being worked with. Packages that will be used within Python are as follows:
 - Pandas and Numpy for preparation and analysis
 - Seaborn and Matplotlib for data visualization
 - Scikit-learn for machine learning and predictive modeling
 - Scipy for statistics and normalization

Part III: Data Preparation

C. Perform data preparation for the chosen data set by doing the following:

1. Describe one data preprocessing goal relevant to the prediction method from part A1.
 - The goal is to have a clean data set that can be used for Decision Tree mentioned in above. In order to ensure the data set is properly prepared for Decision Tree, any

categorical variables will need encoded to numeric.

2. Identify the initial data set variables that you will use to perform the analysis for the prediction question from part A1 and group each variable as numeric or categorical.

- Dependent variable(X)
 - Tenure(numeric)
- Independent variables(Y)
 - Children(numeric)
 - Age(numeric)
 - Income(numeric)
 - Outage_sec_perweek(numeric)
 - Email(numeric)
 - Contacts(numeric)
 - Yearly_equip_failure(numeric)
 - Techie(categorical)
 - MonthlyCharge(numeric)
 - Bandwidth_GB_Year(numeric)

3. Explain each of the steps used to prepare the data for the analysis. Identify the code segment for each step.

- needed packages will be imported into Python
- pd.read_csv will be used to load data frame
- .info() will be used to profile data frame
- inspect data frame for duplicate values
- .isnull().sum will be used to identify possible missing values
- if missing values exist, they will be treated with imputation
- identify outliers
- encoding of categorical variables to dummy variables
- save prepared data to csv

4. Provide a copy of the cleaned data set.

- Code for C3 & C4 is below.

```
In [1]: # code for c3
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import scipy.stats as stats
import sklearn
import warnings
warnings.filterwarnings('ignore', category=FutureWarning)

# code for data frame
churn = pd.read_csv("C:/users/jjord/Documents/WGU/D209/PA2/Rev3/churn_clean.csv")
```

```
# Profile of Data Frame  
churn.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 50 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   CaseOrder             10000 non-null  int64
 1   Customer_id           10000 non-null  object
 2   Interaction            10000 non-null  object
 3   UID                   10000 non-null  object
 4   City                  10000 non-null  object
 5   State                 10000 non-null  object
 6   County                10000 non-null  object
 7   Zip                   10000 non-null  int64
 8   Lat                   10000 non-null  float64
 9   Lng                   10000 non-null  float64
10   Population            10000 non-null  int64
11   Area                  10000 non-null  object
12   TimeZone              10000 non-null  object
13   Job                   10000 non-null  object
14   Children              10000 non-null  int64
15   Age                   10000 non-null  int64
16   Income                10000 non-null  float64
17   Marital               10000 non-null  object
18   Gender                10000 non-null  object
19   Churn                 10000 non-null  object
20   Outage_sec_perweek    10000 non-null  float64
21   Email                 10000 non-null  int64
22   Contacts              10000 non-null  int64
23   Yearly_equip_failure  10000 non-null  int64
24   Techie                10000 non-null  object
25   Contract              10000 non-null  object
26   Port_modem            10000 non-null  object
27   Tablet                10000 non-null  object
28   InternetService       7871 non-null   object
29   Phone                 10000 non-null  object
30   Multiple              10000 non-null  object
31   OnlineSecurity        10000 non-null  object
32   OnlineBackup          10000 non-null  object
33   DeviceProtection      10000 non-null  object
34   TechSupport           10000 non-null  object
35   StreamingTV           10000 non-null  object
36   StreamingMovies       10000 non-null  object
37   PaperlessBilling      10000 non-null  object
38   PaymentMethod         10000 non-null  object
39   Tenure                10000 non-null  float64
40   MonthlyCharge         10000 non-null  float64
41   Bandwidth_GB_Year     10000 non-null  float64
42   Item1                 10000 non-null  int64
43   Item2                 10000 non-null  int64
44   Item3                 10000 non-null  int64
45   Item4                 10000 non-null  int64
46   Item5                 10000 non-null  int64
47   Item6                 10000 non-null  int64
48   Item7                 10000 non-null  int64
49   Item8                 10000 non-null  int64

```

dtypes: float64(7), int64(16), object(27)

memory usage: 3.8+ MB

```
In [2]: # Missing values using isnull  
churn.isnull().sum()
```

```

Out[2]: CaseOrder      0
        Customer_id    0
        Interaction     0
        UID             0
        City            0
        State           0
        County          0
        Zip             0
        Lat             0
        Lng             0
        Population      0
        Area            0
        TimeZone        0
        Job             0
        Children        0
        Age             0
        Income          0
        Marital         0
        Gender          0
        Churn           0
        Outage_sec_perweek 0
        Email           0
        Contacts        0
        Yearly_equip_failure 0
        Techie          0
        Contract        0
        Port_modem      0
        Tablet          0
        InternetService 2129
        Phone           0
        Multiple        0
        OnlineSecurity  0
        OnlineBackup    0
        DeviceProtection 0
        TechSupport     0
        StreamingTV     0
        StreamingMovies 0
        PaperlessBilling 0
        PaymentMethod   0
        Tenure          0
        MonthlyCharge   0
        Bandwidth_GB_Year 0
        Item1           0
        Item2           0
        Item3           0
        Item4           0
        Item5           0
        Item6           0
        Item7           0
        Item8           0
        dtype: int64

```

```

In [3]: # Look at unique values for InternetService
        churn.InternetService.unique()

```

```
Out[3]: array(['Fiber Optic', 'DSL', nan], dtype=object)
```

```
In [4]: # Impute nulls for InternetService
churn["InternetService"].fillna("No Internet Service", inplace=True)
```

```
In [5]: # verify InternetService no longer has nulls
churn["InternetService"].isnull().sum()
```

```
Out[5]: 0
```

Columns Zip, Lat, Lng currently are floats or ints. There are not quantitative so will change datatype to object. These values are not meant for any type of calculation. Since the rest of the data types for categorical are set to object, these will also.

```
In [6]: obj_columns = ["Zip", "Lat", "Lng"]
churn[obj_columns] = churn[obj_columns].astype(object)
```

```
In [7]: # rename item columns
churn.rename(
    columns={
        "Item1": "Timely_response",
        "Item2": "Timely_fixes",
        "Item3": "Timely_replacements",
        "Item4": "Reliability",
        "Item5": "Options",
        "Item6": "Respectful_response",
        "Item7": "Courteous_exchange",
        "Item8": "Active_listening",
    },
    inplace=True,
)
```

```
In [8]: #profile df after clean
churn.info()
```



```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 50 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   CaseOrder             10000 non-null  int64
 1   Customer_id           10000 non-null  object
 2   Interaction            10000 non-null  object
 3   UID                   10000 non-null  object
 4   City                  10000 non-null  object
 5   State                 10000 non-null  object
 6   County                10000 non-null  object
 7   Zip                   10000 non-null  object
 8   Lat                   10000 non-null  object
 9   Lng                   10000 non-null  object
10   Population            10000 non-null  int64
11   Area                  10000 non-null  object
12   TimeZone              10000 non-null  object
13   Job                   10000 non-null  object
14   Children              10000 non-null  int64
15   Age                   10000 non-null  int64
16   Income                10000 non-null  float64
17   Marital               10000 non-null  object
18   Gender                10000 non-null  object
19   Churn                 10000 non-null  object
20   Outage_sec_perweek    10000 non-null  float64
21   Email                 10000 non-null  int64
22   Contacts              10000 non-null  int64
23   Yearly_equip_failure  10000 non-null  int64
24   Techie                10000 non-null  object
25   Contract              10000 non-null  object
26   Port_modem            10000 non-null  object
27   Tablet                10000 non-null  object
28   InternetService       10000 non-null  object
29   Phone                 10000 non-null  object
30   Multiple              10000 non-null  object
31   OnlineSecurity        10000 non-null  object
32   OnlineBackup          10000 non-null  object
33   DeviceProtection      10000 non-null  object
34   TechSupport           10000 non-null  object
35   StreamingTV           10000 non-null  object
36   StreamingMovies       10000 non-null  object
37   PaperlessBilling      10000 non-null  object
38   PaymentMethod         10000 non-null  object
39   Tenure                10000 non-null  float64
40   MonthlyCharge         10000 non-null  float64
41   Bandwidth_GB_Year    10000 non-null  float64
42   Timely_response       10000 non-null  int64
43   Timely_fixes          10000 non-null  int64
44   Timely_replacements   10000 non-null  int64
45   Reliability           10000 non-null  int64
46   Options               10000 non-null  int64
47   Respectful_response   10000 non-null  int64
48   Courteous_exchange    10000 non-null  int64
49   Active_listening      10000 non-null  int64

```

dtypes: float64(5), int64(15), object(30)
memory usage: 3.8+ MB

In [9]: `# code for outliers.
churn.describe()`

Out[9]:

	CaseOrder	Population	Children	Age	Income	Outage_sec_perweek
count	10000.00000	10000.000000	10000.0000	10000.000000	10000.000000	10000.000000
mean	5000.50000	9756.562400	2.0877	53.078400	39806.926771	10.000000
std	2886.89568	14432.698671	2.1472	20.698882	28199.916702	2.000000
min	1.00000	0.000000	0.0000	18.000000	348.670000	0.000000
25%	2500.75000	738.000000	0.0000	35.000000	19224.717500	8.000000
50%	5000.50000	2910.500000	1.0000	53.000000	33170.605000	10.000000
75%	7500.25000	13168.000000	3.0000	71.000000	53246.170000	11.000000
max	10000.00000	111850.000000	10.0000	89.000000	258900.700000	21.000000

In [10]: `#create separate dataframes for numeric & categorical variables
churn_con = churn[['Children', 'Age', 'Income', 'Outage_sec_perweek', 'Email', 'Contract_type', 'Techie'])
churn_cat = churn[['Techie']]

#replace yes & no with 1 & 0
churn_cat = churn_cat.replace({'No': 0, 'Yes': 1})

#one hot encode categorical, dropping first level of each
churn_cat_encode = pd.get_dummies(churn_cat, drop_first=True)

#column concatenate
data = pd.concat([churn_con, churn_cat_encode], axis=1)`

In [11]: `#c4 prepped data set
data.to_csv('prep_churn.csv', index=False)`

Part IV: Analysis

D. Perform the data analysis and report on the results by doing the following:

1. Split the data into training and test data sets and provide the file(s).

In [12]: `# d1 code
from sklearn.model_selection import train_test_split

x = data.drop('Tenure', axis=1)
y = data['Tenure']`

```
#split into test and train
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_st
#files to csv
x_train.to_csv('x_train.csv', index=False)
x_test.to_csv('x_test.csv', index=False)
y_train.to_csv('y_train.csv', index=False)
y_test.to_csv('y_test.csv', index=False)

#files will be attached to submission
```

D

2. Describe the analysis technique you used to appropriately analyze the data. Include screenshots of the intermediate calculations you performed.

- The test_train_split function was used from the sklearn library and was split into testing and training using a 80/20 split(see above). DecisionTreeRegressor will be imported from the sklearn library. Next, the model will be fit with training predictor variable data and dependent data. Once that is done, MSE and R-squared scores will be printed.
- A Decision tree is a tree-like structure that represents a set of decisions and their possible consequences. Each node in the tree represents a decision, and each branch represents an outcome of that decision. The leaves of the tree represent the final decisions or predictions.(GeeksforGeeks, 2024)

```
In [13]: #package import for mse and decision tree regressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import r2_score, accuracy_score, mean_squared_error as mse

#define parameters
p_dtr = {
    'max_depth': [1,2,3,4,5,6,7,8,9,10],
    'min_samples_leaf': [10,25,50,75,100]
}
#decision tree regressor
dtr = DecisionTreeRegressor(random_state=42)

#gridserchcv
grid_dtr = GridSearchCV(estimator=dtr, param_grid=p_dtr, scoring='r2', cv=10, n_job

#fit gridsearch to train
grid_dtr.fit(x_train, y_train)

#find best estimator
best_est = grid_dtr.best_estimator_

# prediction on test
y_pred = best_est.predict(x_test)

#calculate and print
test_mse = mse(y_test, y_pred)
```

```
print('Test set Mean Squared Error: {:.3f}'.format(test_mse))

test_r2 = r2_score(y_test, y_pred)
print('Test set R-squared: {:.3f}'.format(test_r2))

#best parameters
print(grid_dtr.best_params_)
```

Test set Mean Squared Error: 7.933

Test set R-squared: 0.989

{'max_depth': 8, 'min_samples_leaf': 25}

D2 Cont.

- The code above is an attempt at DecisionTreeRegressor optimization. GridSearchCV was used to help find the optimal set for model performance. Best_estimator was used to find the best fit for the model. These steps used above can be considered intermediate calculations for the final model.

D3. Code is below for final model.

In [14]: *#D3 final model*

```
dtr_final = DecisionTreeRegressor(max_depth=8, min_samples_leaf=25, random_state=1)

#model fit
dtr_final.fit(x_train, y_train)

#test label prediction
y_pred = dtr_final.predict(x_test)

#print mean squared
test_mse = mse(y_test, y_pred)
print('Test set MSE: {:.3f}'.format(test_mse))

#print r-squared
test_r2 = r2_score(y_test, y_pred)
print('Test set R-squared: {:.3f}'.format(test_r2))
```

Test set MSE: 7.933

Test set R-squared: 0.989

Part V: Data Summary and Implications

E. Summarize your data analysis by doing the following:

1. Explain the accuracy and the mean squared error (MSE) of your prediction model.
 - The MSE of the model was 7.933. MSE works by finding the average squared difference between the predictions and the observations.(GeeksforGeeks, Retrieving node MSE in

decisiontreeregressor 2024) A lower MSE means the more accurate the model is. With the MSE of this model being 7.933, that would be considered very low accuracy.

2. Discuss the results and implications of your prediction analysis.

- The R-squared of the model was .989 or 98.9%. This means the model predicts the variability of the target variable based on the independent variables used as well as an optimal leaf sample of 25 and an optimal max depth of 8. There is also a concern of over-fitting which is a limitation of decision trees.

3. Discuss one limitation of your data analysis.

- The main limitation of the analysis is that decision trees are prone to over-fitting. Over-fitting occurs when decision trees become too complex to interpret. The decision tree predicts well for the training data but can be inaccurate for new data.(Ravindran, 2023) This is a concern because we don't know if the models performance is due to overfitting rather than showing genuine relationships.

4. Recommend a course of action for the real-world organizational situation from part A1 based on your results and implications discussed in part E2.

- While there is some statistical certainty in the models ability to identify factors that contribute to a customer's tenure, it would be recommended further analysis be done using different variables or even one of the other suggested prediction methods due to the possibility of over-fitting. It is also a recommendation to implement a technique known as pruning if a decision tree is going to be used for further analysis. After further analysis has been done, the company can then make an informed decision on the research question.

Part VI: Demonstration

F. Provide a Panopto video recording that includes the presenter and a vocalized demonstration of the functionality of the code used for the analysis of the programming environment.

- <https://wgu.hosted.panopto.com/Panopto/Pages/Viewer.aspx?id=aedd2ec-b10d-4adb-a91f-b31801487607>

G. List the web sources used to acquire data or segments of third-party code to support the application. Ensure the web sources are reliable.

- none used

H. Acknowledge sources, using in-text citations and references, for content that is quoted, paraphrased, or summarized.

WGU. (n.d.). Churn Data Consideration and Dictionary. Data Files and Associated Dictionary Files.

<https://access.wgu.edu/ASP3/aap/content/d9rkejv84kd9rk30fi2l.zip>

GeeksforGeeks. (2024, May 14). Python: Decision tree implementation. <https://www.geeksforgeeks.org/machine-learning/decision-tree-implementation-python/>

Ttcbest. (2023, October 8). Decision tree (assumptions, advantages, disadvantages, applications). Medium.

<https://medium.com/@ttc4best/decision-tree-assumptions-advantages-disadvantages-applications-46f7e5807517>

GeeksforGeeks. (2024b, September 12). Retrieving node MSE in decisiontreeregressor. <https://www.geeksforgeeks.org/machine-learning/retrieving-node-mse-in-decisiontreeregressor/>

Ravindran, R. (2023, January 31). Overfitting and pruning in decision trees - improving model's accuracy. Medium.

<https://medium.com/nerd-for-tech/overfitting-and-pruning-in-decision-trees-improving-models-accuracy-fdbe9ecd1160>

GeeksforGeeks. (2025, June 2). Decision tree regression using sklearn - python. <https://www.geeksforgeeks.org/machine-learning/python-decision-tree-regression-using-sklearn/>